LAW ERP · FREEBUFF DESKTOP

Case Management Module

Matter → Court Case → Court Event — design, business logic, API reference & frontend guide

Module: `com.lawfirm.erp.modules.casemanagement`

Base path: `/api/v1/firm` · **Auth:** Bearer JWT (firm-scoped, roles FIRM_ADMIN / ADVOCATE / PARALEGAL)

Audience: backend developers, frontend developers, product owners

Version: v1 (Matter/CourtCase/CourtEvent redesign)

1 · Overview & Architecture

The module models a law-firm dispute the way it really moves through the courts. A dispute is a **Matter**; every court instance it is registered in is a **CourtCase**; every diary date (Tarik) or hearing (Peshi) is a **CourtEvent**. The three layers form a chain that can be traced end-to-end: District → High → Supreme, remands back down, writs, revisions, reviews.

Entities

Entity	Meaning
`Matter`	The dispute. One record, forever. Owns the CourtCase chain.
`CourtCase`	One row per court instance (per level/registration). Linked by `parentCourtCaseId`.
`CourtEvent`	One row per Tarik/Peshi. Sequenced per CourtCase; chained by `nextEventId`.
`MatterParty`	Party identity at Matter level (dedup-able).
`CourtCaseRole`	Party's role on a specific CourtCase (roles flip on appeal).
`MatterTimelineEvent`	Immutable event log entry, tagged with matter + optional court case.
`AppealDeadlineRule`	Statutory appeal window per level/type/state-party.

Cross-cutting conventions

- **Multi-tenancy:** every row carries `firmId`; firm context comes from the JWT / subdomain (`FirmContextHolder`).
- **Audit:** every mutation writes an audit log (`AuditService`) and a timeline event.
- **Envelope:** all responses wrap in `ApiResponse`, all request bodies in `ApiRequest`.
- **Ids:** UUIDs; human references are generated (`matterNumber`, `ourCourtCaseRef`).
- **Soft state:** entities extend `ActiveAuditableEntity` (is_active, createdAt/updatedAt).

> **Key idea — one chain per dispute.** A dispute that goes District → High → Supreme is *one Matter* with *three CourtCase rows* (ORIGINAL, APPEAL, FURTHER_APPEAL). The Matter always points at its current leaf via `currentCourtCaseId` — the row the UI should open by default.

2 · Business Logic — how everything processes

2.1 Create a Matter

1. Server generates `matterNumber` (`{firmCode}-MAT-{year}-{n}`, e.g. `APX-MAT-2026-00001`).
2. A `Matter` row is created (status `ACTIVE`).
3. An **ORIGINAL** `CourtCase` is created at the originating court level with its own ref (`{matterNumber}-DC1` etc.) and the initial stage for that level/type.
4. Parties are created as `MatterParty` and given a `CourtCaseRole` on the original case.
5. The Matter's `currentCourtCaseId` is set to the new case; timeline + audit entries are written.

> **Ref format:** internal court-case ref is always populated — the official `courtCaseNumber` may be null until the registry assigns it.

2.2 The CourtCase chain (appeal / remand / writ / revision / review)

`RelationType` decides how the new instance attaches and how the parent is marked:

Relation	Parent handling	Notes
`ORIGINAL`	—	Only created via create-matter.
`APPEAL` / `CROSS_APPEAL` / `REVISION` / `REVIEW`	parent → `APPEALED`	Roles are created fresh (appellant/respondent).
`REMAND`	parent → `REMANDED`	Case comes back to a lower level; level counter increments (DC2).
`WRIT`	no parent	Cannot be filed at District level (business rule).

- The child's ref = `{matterNumber}-{levelCode}{n}` where `n` counts instances at that level (DC1, HC1, DC2 after remand, SC1…).
- The Matter leaf (`currentCourtCaseId`) moves to the child; matter returns to `ACTIVE` .
- You cannot attach a proceeding to a closed parent.

2.3 Parties & roles

- **Identity** lives on `MatterParty` (name, phone, email, address, optional linked `clientId`).
- **Role** lives on `CourtCaseRole` (PLAINTIFF/DEFENDANT/ACCUSED/APPELLANT/RESPONDENT/APPLICANT + REPRESENTED/OPPOSING/SELF + per-instance advocate).
- On appeal the roles flip — the District defendant becomes the High-Court appellant — which is why roles are per-instance.
- **Party matching** (`POST /matters/parties/match`) dedupes a new party against existing clients and matter parties, ranked HIGH / MEDIUM / LOW by name/phone/email overlap.

2.4 The Tarik/Peshi loop (CourtEvent)

This is the digital diary. An event is `TARIK` (administrative date) or `PESHI` (actual hearing).

1. **Schedule:** create an event with date/time, courtroom, judge, attending advocate. If an advocate + time is given, a firm-wide conflict check runs and rejects overlaps.
2. **Mark held:** `POST /court-events/{id}/held` requires the answer to *"what did the court give next?"*:
   - `nextEventType=TARIK|PESHI` → the server creates the next event row and links it via `nextEventId` — the loop continues.
   - `nextEventType=JUDGMENT` → no event row; judgment is recorded via the judgment endpoint.
   - `nextEventType=NONE` → the matter is closed at this event.
3. **Outcome side-effects:**
   - `JUDGMENT_DELIVERED` at stage `JUDGMENT_AWAITED` → court case becomes `DECIDED` .
   - `WITHDRAWN` → case status `WITHDRAWN` .
   - `ADJOURNED_NO_PROGRESS` → event marked `ADJOURNED` .

2.5 Stage machine

Stages are level/type/relation aware (`CourtCaseStage`): civil flow (FILED → SUMMONS_ISSUED → RESPONSE_PENDING → MEDIATION → HEARING_STAGE → JUDGMENT_AWAITED → JUDGMENT_DELIVERED → EXECUTION/CLOSED), criminal flow (FIR_REGISTERED → UNDER_INVESTIGATION → CHARGE_SHEET_FILED → … → SENTENCING), appeal flow (APPEAL_FILED → ADMITTED → … → FURTHER_APPEALED), writ flow (WRIT_FILED → NOTICE_ISSUED → ORDER_ISSUED).

- `updateStage` validates: not terminal, valid for level/type/relation, and in `allowedTransitions()` .
- Special moves: `EXECUTION` clears `appealLapsed` ; leaving `MEDIATION` logs MEDIATION_SUCCEEDED/FAILED; `CLOSED` closes the case.

2.6 Judgment & appeal deadlines (AppealDeadlineEngine)

Recording a judgment computes the statutory appeal window from seeded rules (NCPC 2074):

Appealed from	Type	State party	Window	Extension
District	Civil	—	30 days	+15

District	Criminal	yes	70 days	+30
District	Criminal	no	30 days	+30
High	Civil	—	35 days	+15
High	Criminal	—	35 days	+30
High	Criminal	yes	70 days	+30

- A nightly scheduled job (`closeLapsedAppeals` , 03:00) finds `DECIDED` cases whose window lapsed with no appeal filed and no child case → marks `appealLapsed=true` (judgment final) and nudges the Matter to `DORMANT` .
- If the firm later moves the case to `EXECUTION` , `appealLapsed` is cleared.

2.7 Stale matters

`GET /matters/stale` lists matters whose current leaf hasn't had a real (non-cancelled) Peshi in N days (default 90) — the long-pending Tarik chains — with `daysSinceLastPeshi` .

2.8 Timeline — per matter and overall

- **Per matter:** `GET /matters/{matterNumber}/timeline` — every event across all of the matter's court cases, newest first, each tagged with its `ourCourtCaseRef` .
- **Overall (firm-wide):** `GET /matters/timeline` — the aggregate activity feed of *every* matter in the firm, newest first, filterable by matter type, matter status, and date window. This is the "everything working" overview screen. Events carry `matterNumber` , `matterTitle` , `ourCourtCaseRef` and `eventType` so the UI can render a grouped, filterable feed.

Every meaningful mutation writes a `MatterTimelineEvent` : MATTER_CREATED, COURT_CASE_ADDED, STAGE_CHANGE, MEDIATION_FAILED/SUCCEEDED, COURT_EVENT_SCHEDULED/HELD/ADJOURNED/CANCELLED, PARTY_ADDED, JUDGMENT_RECORDED, APPEAL_FILED, MATTER_NOTE_ADDED.

2.9 Calendar

The calendar is a read-only projection of `CourtEvent` (the single source of truth) across all court cases: date range, today, and upcoming N days, optionally filtered by advocate.

3 · API Reference

3.0 Conventions

Base: `/api/v1/firm` · **Headers:** `Authorization: Bearer <JWT>` (+ firm context via subdomain/firm code).

Request envelope: `POST/PUT` bodies are `{ "data": { … } }` .

```
// Success envelope
{ "success": true, "responseCode": 200, "message": "Matter created successfully", "data": { … } }

// Error envelope (HTTP status matches the error type)
{ "success": false, "responseCode": 400, "message": "Cannot transition from 'FILED' to 'CLOSED'", "data": null }
```

Pagination: list endpoints return a Spring `Page` : `content[], totalElements, totalPages, number, size, first, last, empty, sort` .

Endpoint map

Method	Path	Purpose
POST	`/matters`	Create matter + original court case + parties

GET	`/matters`	List matters (filters: matterType, status, search)
GET	`/matters/{matterNumber}`	Full matter detail (chain, roles, parties)
PUT	`/matters/{matterNumber}`	Update matter details/status
GET	`/matters/{matterNumber}/timeline`	Timeline for one matter
GET	`/matters/timeline`	**Overall firm-wide timeline** (filters: matterType, status, from, to)
GET	`/matters/stale`	Matters with no real Peshi in N days
POST	`/matters/{matterNumber}/court-cases`	Attach appeal/remand/writ/review court case
POST	`/matters/{matterNumber}/parties`	Add a party to the matter
POST	`/matters/parties/match`	Dedup match against clients/matter parties
GET	`/court-cases/{ourCourtCaseRef}`	Court case detail
PUT	`/court-cases/{ourCourtCaseRef}`	Update court case fields
PUT	`/court-cases/{ourCourtCaseRef}/stage`	Advance/change stage (validated)
PUT	`/court-cases/{ourCourtCaseRef}/judgment`	Record judgment + appeal deadline
POST	`/court-cases/{ourCourtCaseRef}/events`	Schedule Tarik/Peshi event
GET	`/court-cases/{ourCourtCaseRef}/events`	Event stream for a court case
GET	`/court-events/{eventId}`	Event detail
PUT	`/court-events/{eventId}`	Update event
POST	`/court-events/{eventId}/held`	Mark held → outcome + next event
DELETE	`/court-events/{eventId}`	Cancel event
GET	`/calendar`	Events in a date range (default −30d…+90d)
GET	`/calendar/today`	Today's events
GET	`/calendar/upcoming`	Next N days (1–365)

3.1 Matters

POST `/api/v1/firm/matters` — create a matter

Creates the Matter, its ORIGINAL CourtCase at the originating level, and the parties (each with a role on the original case).

```
{
  "data": {
    "matterType": "CIVIL",
    "title": "Ram vs Shyam — land boundary dispute",
    "originatingCourtLevel": "DISTRICT",
    "courtName": "Kathmandu District Court",
    "courtCaseNumber": "C-123/082",
    "filingDate": "2026-08-01",
    "assignedPartnerId": "…uuid…",
    "advocateId": "…uuid…",
    "description": "Boundary dispute over plot no. 45",
    "parties": [
      { "fullName": "Ram Sharma", "mobileNo": "9800000001", "email": "ram@mail.com",
        "clientId": "…uuid…", "isOurClient": true, "roleType": "PLAINTIFF", "representation": "REPRESENTED" },
      { "fullName": "Shyam Thapa", "mobileNo": "9800000002", "roleType": "DEFENDANT", "representation": "OPPOSING" }
    ]
  }
}
```

Returns: `MatterResponse` (full: court cases + roles + parties). Note the generated `matterNumber` and `ourCourtCaseRef` — save them; they are the primary lookups for every other call.

GET `/api/v1/firm/matters?matterType=&status=&search=&page=&size=` — list matters

Paginated, filtered. `search` matches title or matter number (case-insensitive). List mode returns summary (no nested chain).

GET `/api/v1/firm/matters/{matterNumber}` — matter detail

Full tree: all `courtCases[]` (each with `roles[]`, `eventCount`, refs, stage/status, judgment fields, criminal/civil fields), plus `parties[]` and `currentCourtCaseId`. Use this for the matter detail screen.

PUT `/api/v1/firm/matters/{matterNumber}` — update matter

Partial update: `title, description, assignedPartnerId, status` (ACTIVE/DORMANT/CLOSED).

GET `/api/v1/firm/matters/{matterNumber}/timeline` — one matter's timeline

```
{
  "success": true, "responseCode": 200, "message": "Timeline fetched successfully",
  "data": [
    { "id": "…", "matterId": "…", "matterNumber": "APX-MAT-2026-00001", "matterTitle": "Ram vs Shyam…",
      "courtCaseId": "…", "ourCourtCaseRef": "APX-MAT-2026-00001-DC1",
      "eventType": "COURT_EVENT_HELD", "title": "Event held: …-DC1 #3", "description": "…",
      "createdAt": "2026-08-05T14:32:00", "createdBy": "…uuid…" }
  ]
}
```

GET `/api/v1/firm/matters/timeline?matterType=&status=&from=&to=&page=&size=` — **overall timeline**

Firm-wide activity feed across all matters (newest first). `from` / `to` are `YYYY-MM-DD`. Response items are identical to the per-matter timeline above (including `matterTitle`), paginated. Recommended for the dashboard / "everything working" overview.

GET `/api/v1/firm/matters/stale?days=&page=&size=` — stale matters

Matters whose current leaf has no real Peshi in `days` (default 90). Item shape: `{ id, matterNumber, title, matterType, status, currentCourtCaseRef, daysSinceLastPeshi }`.

POST `/api/v1/firm/matters/{matterNumber}/court-cases` — attach a court case

```
{
  "data": {
    "relationType": "APPEAL",
    "courtLevel": "HIGH",
    "courtName": "High Court, Patan",
    "courtCaseNumber": "HC-555/083",
    "filingDate": "2026-08-10",
    "advocateId": "…uuid…",
    "judgeName": "Justice A. Poudel",
    "parentCourtCaseId": "…uuid of the DC1 case…",   // optional — defaults to current leaf
    "partyIsState": false,
    "roles": [
      { "matterPartyId": "…uuid…", "roleType": "APPELLANT", "representation": "REPRESENTED" },
      { "matterPartyId": "…uuid…", "roleType": "RESPONDENT", "representation": "OPPOSING" }
    ]
  }
}
```

New parties may be created inline by sending `fullName/mobileNo/email/clientId` instead of `matterPartyId`. `WRIT` at `DISTRICT` is rejected. The parent is marked APPEALED/REMANDED and the matter leaf moves.

POST `/api/v1/firm/matters/{matterNumber}/parties` — add party

Body: `PartyEntryRequest` (same shape as the party objects above, including `roleType` / `representation`). The party is also given a role on the current leaf court case.

POST `/api/v1/firm/matters/parties/match` — party dedup

```
{ "data": { "fullName": "Ram Sharma", "mobileNo": "9800000001", "email": "ram@mail.com" } }

→ data: [ { "sourceType": "CLIENT", "sourceId": "…", "fullName": "…", "mobileNo": "…", "email": "…", "confidence": "HIGH" }
          { "sourceType": "MATTER_PARTY", "sourceId": "…", "caseNumber": "APX-MAT-2026-00001", "fullName": "…", "confidence
```

3.2 Court cases

GET `/api/v1/firm/court-cases/{ourCourtCaseRef}` — court case detail

Ref format: `APX-MAT-2026-00001-DC1`. Returns the full `CourtCaseResponse` (same shape as the nested objects in matter detail): ref, relation, level, court, filing date, stage/status, advocate/judge, judgment + appeal fields, criminal/civil fields, roles, eventCount.

PUT `/api/v1/firm/court-cases/{ourCourtCaseRef}` — update fields

Partial: `courtCaseNumber, courtName, advocateId, judgeName` + criminal fields (`firNumber, firDate, policeStation, investigationAuthority, arrestDate, chargeSheetDate, bailStatus`) + civil fields (`mediationDate, mediationOutcome, writtenStatementDeadline`).

PUT `/api/v1/firm/court-cases/{ourCourtCaseRef}/stage` — change stage

```
{ "data": { "stage": "JUDGMENT_AWAITED" } }
```

Validates legality + transitions. Rejects changes on terminal (closed) cases.

PUT `/api/v1/firm/court-cases/{ourCourtCaseRef}/judgment` — record judgment

```
{
  "data": {
    "judgmentDate": "2026-09-01",
    "judgmentSummary": "Boundary fixed in favor of the plaintiff…",
    "decisionInFavorOfPartyId": "…uuid of MatterParty…",
    "partyIsState": false
  }
}
```

Moves the case to `JUDGMENT_DELIVERED` / `DECIDED`, clears `appealLapsed`, and computes `appealDeadline` from the seeded statutory rules.

3.3 Court events (Tarik/Peshi)

POST `/api/v1/firm/court-cases/{courtCaseRef}/events` — schedule event

```
{
  "data": {
    "eventType": "PESHI",
    "scheduledDate": "2026-08-15",
    "scheduledTime": "10:30:00",
    "endTime": "11:30:00",
    "attendingAdvocateId": "…uuid…",
    "judgeName": "Judge B. Karki",
    "courtRoom": "Court 3",
    "notes": "Cross-examination of DW-1"
  }
}
```

Sequence number is auto-assigned. Advocate time conflicts (firm-wide) are rejected with a 400 business rule.

GET `/api/v1/firm/court-cases/{courtCaseRef}/events` — event stream

All events in sequence order; each carries `nextEventId` so the frontend can render the Tarik/Peshi chain.

GET `/api/v1/firm/court-events/{eventId}` · **PUT** `/court-events/{eventId}` · **DELETE** `/court-events/{eventId}`

Detail; partial update (`scheduledDate/time, endTime, judgeName, courtRoom, notes, attendingAdvocateId` — re-checks conflicts); cancel (held events cannot be cancelled).

POST `/api/v1/firm/court-events/{eventId}/held` — mark held (the loop)

```
{
  "data": {
    "outcome": "Both sides heard; court adjourned to next hearing",
    "outcomeType": "ADJOURNED_NO_PROGRESS",
    "nextEventType": "PESHI",
    "nextEventDate": "2026-09-02",
    "nextEventTime": "11:00:00",
    "notes": "Next date given in open court"
  }
}
```

- `nextEventType=TARIK|PESHI` + `nextEventDate` (required) → server creates the next event and links it.
- `nextEventType=JUDGMENT` → no next event; record the judgment via the court-case judgment endpoint.
- `nextEventType=NONE` → the case is closed at this event.
- Outcome side-effects: JUDGMENT_DELIVERED/WITHDRAWN update the court-case status as described in §2.4.

3.4 Calendar

`GET` `/api/v1/firm/calendar?from=&to=&advocateId=`

Date range (defaults −30d…+90d). Accepts `YYYY-MM-DD` or ISO datetime. Items: `{ id, courtCaseId, ourCourtCaseRef, matterNumber, matterTitle, eventType, scheduledDate, scheduledTime, endTime, courtRoom, status, attendingAdvocateId }`.

`GET` `/api/v1/firm/calendar/today?advocateId=` · `GET` `/api/v1/firm/calendar/upcoming?days=&advocateId=`

Today's events; next `days` (1–365, default 7).

4 · Frontend consumption guide — step by step

Step 0 — Authentication & context

1. Login via the auth endpoints and store the JWT.
2. Send `Authorization: Bearer <JWT>` on every request; the firm is derived from the token/subdomain — you don't send firmId.
3. Only FIRM_ADMIN, ADVOCATE and PARALEGAL roles can call these endpoints (enforced server-side).

Step 1 — Create a matter (first screen)

1. Optional: call `POST /matters/parties/match` while typing a party name to surface existing clients (HIGH/MEDIUM/LOW) and offer "link to client".
2. Call `POST /matters` with the payload from §3.1.
3. From the response, store `matterNumber` (e.g. `APX-MAT-2026-00001`) and the original case's `ourCourtCaseRef` (e.g. `…-DC1`) — they are the IDs for every later call.

Step 2 — Case diary: schedule events

1. On the court-case screen, call `GET /court-cases/{ref}` to load detail, then `GET /court-cases/{ref}/events` for the existing Tarik/Peshi stream.
2. Schedule the first date: `POST /court-cases/{ref}/events` with `eventType=TARIK|PESHI`, date, time, advocate.
3. On the day: `POST /court-events/{eventId}/held`. The form MUST ask "what did the court give next?" — the next event is created automatically. The event list now shows the chain (`nextEventId`).
4. Update/cancel dates freely with `PUT/DELETE /court-events/{eventId}`.

Step 3 — Judgment & appeal

1. Move the case to `JUDGMENT_AWAITED` (`PUT /court-cases/{ref}/stage`) when arguments close.
2. When the court delivers: `PUT /court-cases/{ref}/judgment` — the response includes the computed `appealDeadline`. Show it on the case header.
3. To appeal: `POST /matters/{matterNumber}/court-cases` with `relationType=APPEAL`, the new level, and roles. The matter detail now shows both cases (DC1 + HC1) and the leaf moves.

Step 4 — Overview screens (read side)

- **Dashboard / firm-wide activity:** `GET /matters/timeline?matterType=CIVIL&status=ACTIVE&from=2026-08-01&to=2026-08-31` — render newest-first with matter number + title + court-case ref + eventType badge + time.
- **Matter detail:** `GET /matters/{matterNumber}` — render the court-case chain as a vertical timeline (ORIGINAL → APPEAL → …), roles per instance, judgment fields.
- **One matter's history:** `GET /matters/{matterNumber}/timeline`.
- **Calendar:** `GET /calendar?from=…&to=…` for month view, `/calendar/today` for the day view, `/calendar/upcoming?days=7` for the widget.
- **Overdue watch:** `GET /matters/stale?days=90` for long-pending matters.

Step 5 — Handling errors & edge cases

- 400 `BusinessRuleException` — e.g. illegal stage transition, advocate conflict, WRIT at District, held event cancel, missing `nextEventDate`.
- 404 `ResourceNotFoundException` — unknown matter number / court-case ref / event id.
- 403 — wrong firm context or insufficient role.
- Always read `message` for user-facing text; keys are human-readable English.

Enum cheat-sheet for dropdowns

Enum	Values
MatterType	CIVIL, CRIMINAL
MatterStatus	ACTIVE, DORMANT, CLOSED
CourtLevel	DISTRICT, HIGH, SUPREME, SPECIALIZED
RelationType	ORIGINAL, APPEAL, CROSS_APPEAL, REMAND, REVISION, WRIT, REVIEW
PartyType	PLAINTIFF, DEFENDANT, ACCUSED, APPELLANT, RESPONDENT, APPLICANT
PartyRepresentation	REPRESENTED, OPPOSING, SELF
CourtEventType	TARIK, PESHI
CourtEventStatus	SCHEDULED, HELD, ADJOURNED, CANCELED
OutcomeType	PART_HEARD, ARGUMENTS_COMPLETE, EVIDENCE_TAKEN, ADJOURNED_NO_PROGRESS, ORDER_PASSED, ORDER_ISSUED, STAY_GRANTED, INTERIM_ORDER, JUDGMENT_DELIVERED, WITHDRAWN
NextEventType	TARIK, PESHI, JUDGMENT, NONE
CourtCaseStatus	ACTIVE, JUDGMENT_AWAITED, DECIDED, APPEALED, REMANDED, CLOSED, WITHDRAWN
CourtCaseStage	civil: FILED, SUMMONS_ISSUED, RESPONSE_PENDING, MEDIATION, HEARING_STAGE, JUDGMENT_AWAITED, JUDGMENT_DELIVERED, EXECUTION; criminal: FIR_REGISTERED, UNDER_INVESTIGATION, CHARGE_SHEET_FILED, SENTENCING; appeals: APPEAL_FILED, ADMITTED, FURTHER_APPEALED; writs: WRIT_FILED, NOTICE_ISSUED, ORDER_ISSUED; shared: APPEALED, REMANDED, CLOSED
TimelineEventType	MATTER_CREATED, COURT_CASE_ADDED, STAGE_CHANGE, MEDIATION_FAILED, MEDIATION_SUCCEEDED, COURT_EVENT_SCHEDULED, COURT_EVENT_HELD, COURT_EVENT_ADJOURNED, COURT_EVENT_CANCELLED, PARTY_ADDED, JUDGMENT_RECORDED, APPEAL_FILED, MATTER_NOTE_ADDED

Case Management Module guide — generated from the implementation in `modules/casemanagement`. Endpoints live under `/api/v1/firm`; Swagger UI is available at `/swagger-ui.html`.